

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.(BL3)*

Assessment Tool Weightage:20%

QUESTIONS: 15

Total Marks:50

Annexure A

Viva Voce

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Viva Voce

QUESTIONS

1. A lexical analyzer reads `for l=10;`. How will it differentiate between a keyword and an identifier? Explain your reasoning.
2. If spaces are removed from a source program, how does the lexical analyzer still recognize tokens correctly?
3. A compiler treats `count@1` as invalid. Analyze why lexical analysis rejects it.
4. Compare how a compiler and interpreter handle lexical errors during execution.
5. If a lexical analyzer fails to identify comments correctly, what problems may occur in later phases?
6. A token can match multiple patterns. How should the lexer decide the correct token?
7. Why is regular expression suitable for token specification? Analyze with an example.
8. How does lexical analysis improve the efficiency of parsing?
9. A language introduces a new keyword. Which lexical rules must change and why?
10. If lexical analysis is skipped, how will syntax analysis be affected?

Syntax Analysis / Parsing (11–20)

11. A parser receives valid tokens but reports an error. Analyze possible reasons.
12. Why is ambiguity considered a major problem in syntax analysis?
13. Compare top-down and bottom-up parsing for arithmetic expressions. Which is better and why?
14. A grammar contains left recursion. Explain how it impacts recursive descent parsing.
15. A parser generates multiple parse trees for one input. How will you resolve it?
16. Why is left factoring required in predictive parsing?
17. If parentheses are missing in an expression, explain how the parser detects it.
18. Evaluate why LL(1) parsing is considered deterministic.

19. How does operator precedence affect parse tree generation?

20. Why is the dangling else problem difficult for predictive parsing?

FIRST / FOLLOW / LL(1) (21–28)

21. If ϵ belongs to FIRST(A), how does it influence FOLLOW(A)?

22. A grammar has overlapping FIRST sets. Analyze the effect on LL(1) parsing.

23. Why is FOLLOW needed only when ϵ appears? Explain with logic.

24. If one cell in an LL(1) table contains two productions, what conclusion can you make?

25. Compare FIRST and FOLLOW with a practical parser example.

26. Why can't every grammar be converted into LL(1)?

27. Analyze how FIRST and FOLLOW help avoid backtracking.

28. A parsing table has an empty entry. What does it indicate?

29. In shift-reduce parsing, how does the parser decide between shift and reduce?

30. Why is handle identification important in bottom-up parsing?

31. A shift-reduce parser enters conflict. How would you analyze and solve it?

32. Compare shift-reduce parsing with predictive parsing in terms of error handling.

33. Why is operator precedence parsing suitable for arithmetic expressions?

34. Analyze a case where reduce-reduce conflict may occur.

35. Why does bottom-up parsing begin from input and move toward the start symbol?

36. Which parsing method is more powerful: LL(1) or LR? Justify.

LR / SLR Parsing (37–42)

37. Why is grammar augmentation necessary before LR parsing?

38. How do closure and goto functions help build LR item sets?

39. Why is SLR more powerful than LR(0)?

40. A parsing table has a shift-reduce conflict. How can FOLLOW sets help resolve it?

41. Compare LR(0), SLR, and LL(1) in terms of grammar handling.

42. Why are LR parsers widely used in compiler construction?

Semantic Analysis / Symbol Table (43–46)

43. A program is syntactically correct but fails compilation. Why does semantic analysis reject it?

44. How does the symbol table support multiple compiler phases?

45. A variable is declared twice in different scopes. How does semantic analysis resolve it?

46. Why is type checking important before code generation?

Intermediate Code / Optimization / Code Generation (47–50)

47. Why is three-address code preferred for optimization?

48. Compare machine-independent and machine-dependent optimization.

49. How does peephole optimization improve target code efficiency?

50. A compiler must choose between faster execution and smaller code size. Which optimization would you prefer and why?

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and	Responds correctly but with	Responds partially; requires	Unable to respond to

		confidently, extending answers with depth	limited depth	prompting	questions effectively
--	--	---	---------------	-----------	--------------------------